

CHAPTER 14.

HTML5의 SVG, 드래그 앤 드롭,



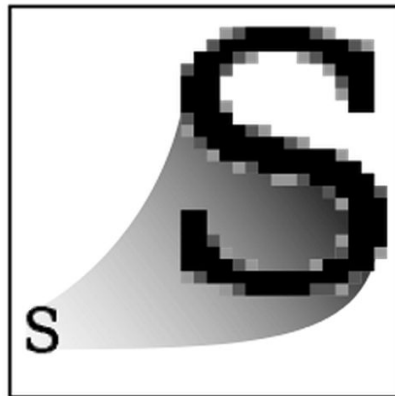
SVG

- **SVG(Scalable Vector Graphics)**는 XML-기반의 벡터 이미지 포맷
- 웹에서 벡터-기반의 그래픽을 정의하는데 사용
- 1999년부터 W3C에 의하여 표준



SVG의 장점

- SVG 그래픽은 확대되거나 크기가 변경되어도 품질이 손상되지 않는다.
- SVG 파일에서 모든 요소와 속성은 애니메이션이 가능하다.
- SVG 이미지는 어떤 텍스트 에디터로도 생성하고 편집할 수 있다.



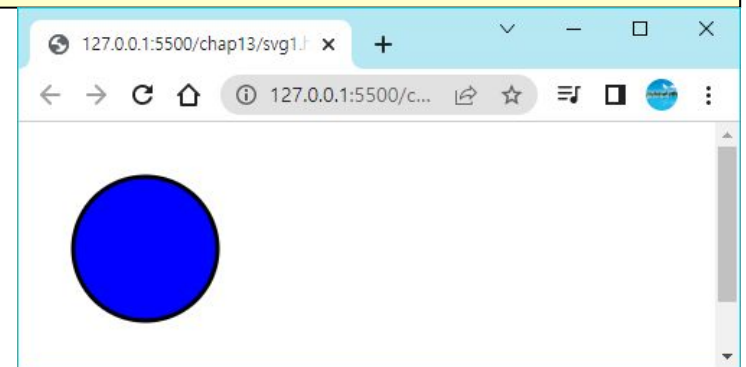
Raster
.jpeg .gif .png



Vector
.svg

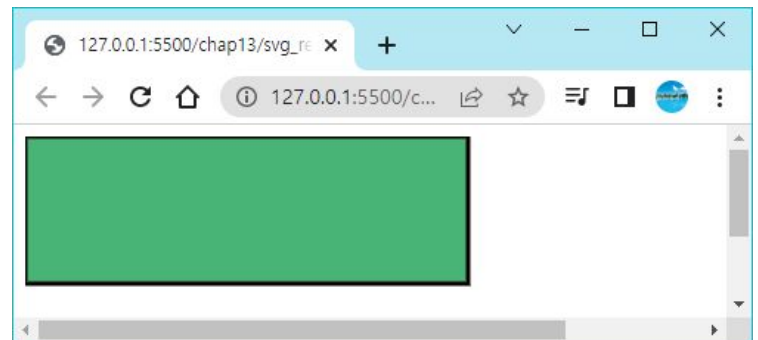
원 예제

```
<!DOCTYPE html>
<html>
<body>
  <svg width="200" height="200">
    <circle cx="80" cy="80" r="50" stroke="black"
      stroke-width="3" fill="blue"/>
  </svg>
</body>
</html>
```



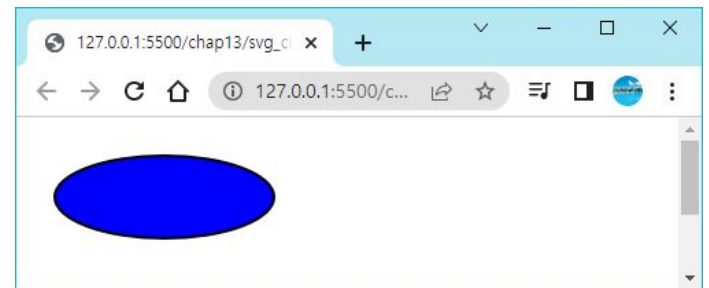
사각형 예제

```
..  
<svg width="600" height="200">  
  <rect width="300" height="100"  
    style="fill: rgb(72, 180, 117);  
      stroke-width: 3; stroke: rgb(12, 0, 0)"/>  
</svg>
```



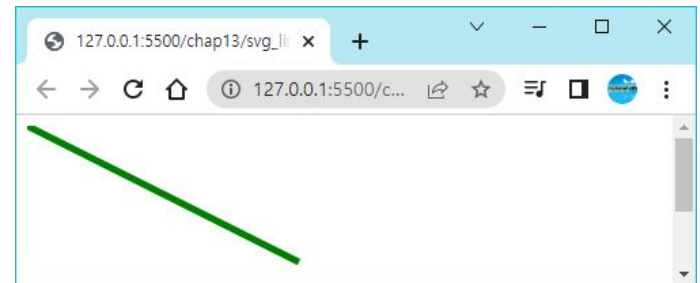
타원 예제

```
<svg width="300" height="200">  
  <ellipse cx="100" cy="50" rx="80" ry="30"  
    style="fill:blue; stroke:black; stroke-width:2;"/>  
</svg>
```



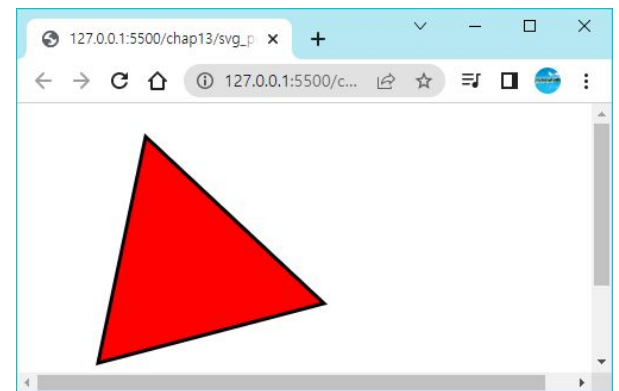
직선 예제

```
<svg width="300" height="200">  
  <line x1="0" y1="0" x2="200" y2="100"  
    style="stroke: green; stroke-width: 5"/>  
</svg>
```



다각형 예제

```
<svg width="500" height="300">  
  <polygon points="100,20 250,160 60,210"  
    style="fill: red; stroke: black; stroke-width: 3"/>  
</svg>
```



텍스트 예제

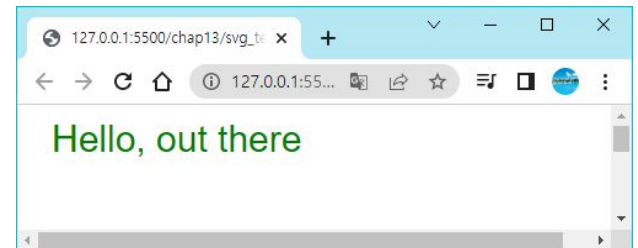
```
<svg width="500" height="300">
```

```
  <text x="20" y="30" font-family="Arial" font-size="30" fill="green">
```

```
    Hello, out there
```

```
  </text>
```

```
</svg>
```

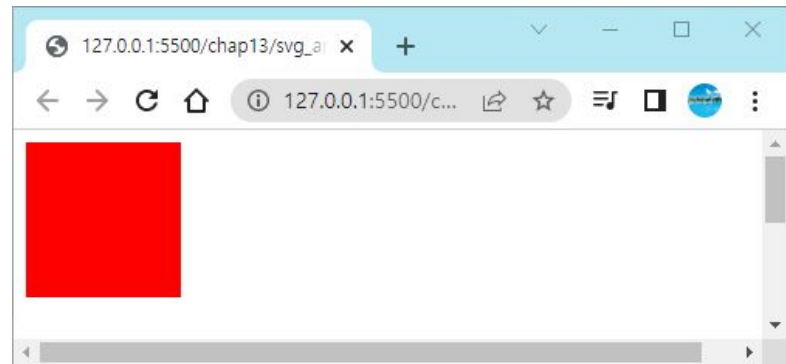


SVG 애니메이션

- `<rect>` 요소 안에 `<animate>`를 추가하였다. `<animate>` 요소는 다음과 같은 속성을 포함한다.
 - `attributeName` - 이 속성은 애니메이션에서 변화되는 속성을 지정한다.
 - `from` - 속성이 시작되는 값을 지정한다.
 - `to` - 속성이 끝나는 값을 지정한다. 이 예제에서는 사각형의 높이는 증가된다.
 - `dur` - 이 속성은 애니메이션의 지속 시간을 지정한다. 예를 들어, `01:30`은 동안 1 분 30 초를 나타낸다. `2s`는 2초, `2h`는 2시간을 의미한다.

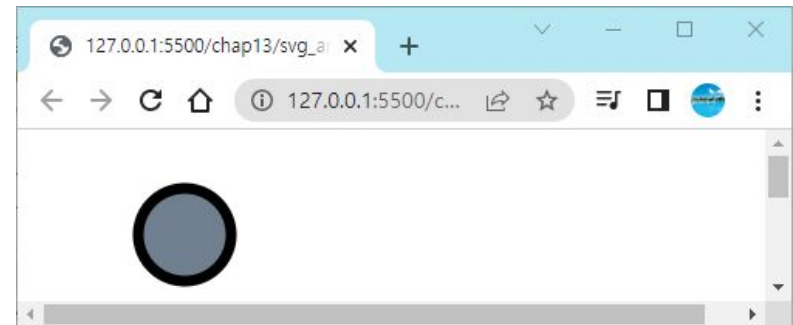
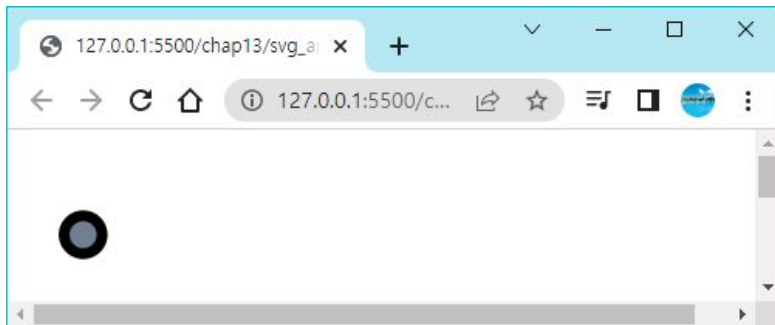
애니메이션 #1

```
<svg width="500" height="300">  
  <rect x=0 y=0 width="100" height="100" fill="red">  
    <animate attributeName="height" from="0" to="100" dur="10s"/>  
  </rect>  
</svg>
```



애니메이션 #2

```
<svg width="500" height="300">  
  <circle r="30" cx="100" cy="60" fill="slategrey" stroke="#000" stroke-width="7">  
    <animate attributeName="r" from="0" to="30" dur="3s"/>  
    <animate attributeName="cx" from="0" to="100" dur="3s"/>  
  </circle>  
</svg>
```



viewBox 속성

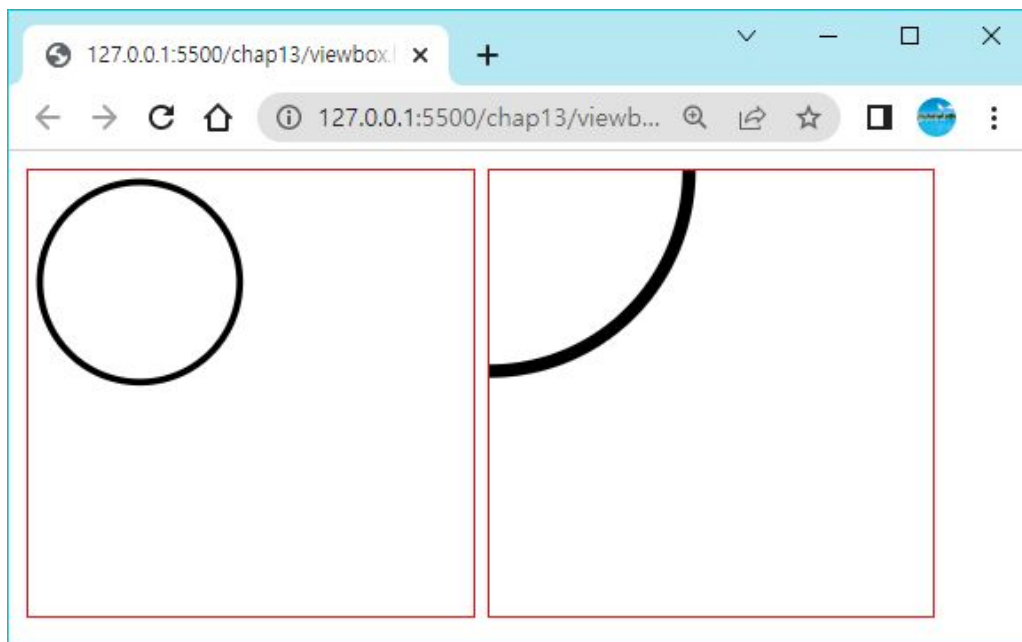
- viewBox는 SVG가 그림을 그리는 영역을 정의하는 속성이다.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    svg {      border: 1px solid rgb(230, 20, 20);    }
  </style>
</head>
<body>

  <svg width="200" height="200" viewBox="0 0 200 200">
    <circle cx="50" cy="50" r="45"
      stroke="#000" stroke-width="3"
      fill="none"/>
  </svg>

  <svg width="200" height="200" viewBox="50 50 100 100">
    <circle cx="50" cy="50" r="45" stroke="#000"
      stroke-width="3" fill="none"/>
  </svg>
</body>
</html>
```

실행 결과



SVG와 캔버스 그래픽의 차이점

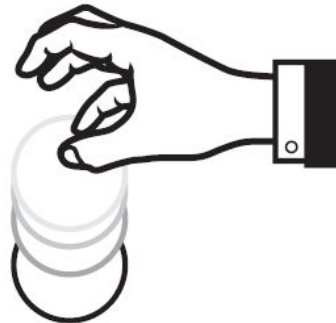
캔버스	SVG
• 해상도에 의존적이다. • 이벤트 핸들러를 붙일 수 없다. • 텍스트는 비교적 낮은 품질로 그려진다. • 이미지를 .png나 .jpg 파일로 저장할 수 있다. • 그래픽을 많이 사용하는 게임에 적합하다.	• 해상도에 독립적이다. • 이벤트 핸들러를 붙일 수 있다. • 구글 지도와 같은 큰 영역을 렌더링하는 경우 적합하다. • 그리기가 복잡하면 속도가 늦어진다. • 게임에는 적합하지 않다.

드래그와 드롭

- **드래그(drag)**와 **드롭(drop)** - 윈도우에서 아주 많이 사용하는 사용자 인터페이스 중의 하나
- 객체를 마우스로 끌어서 다른 애플리케이션에 놓는 것

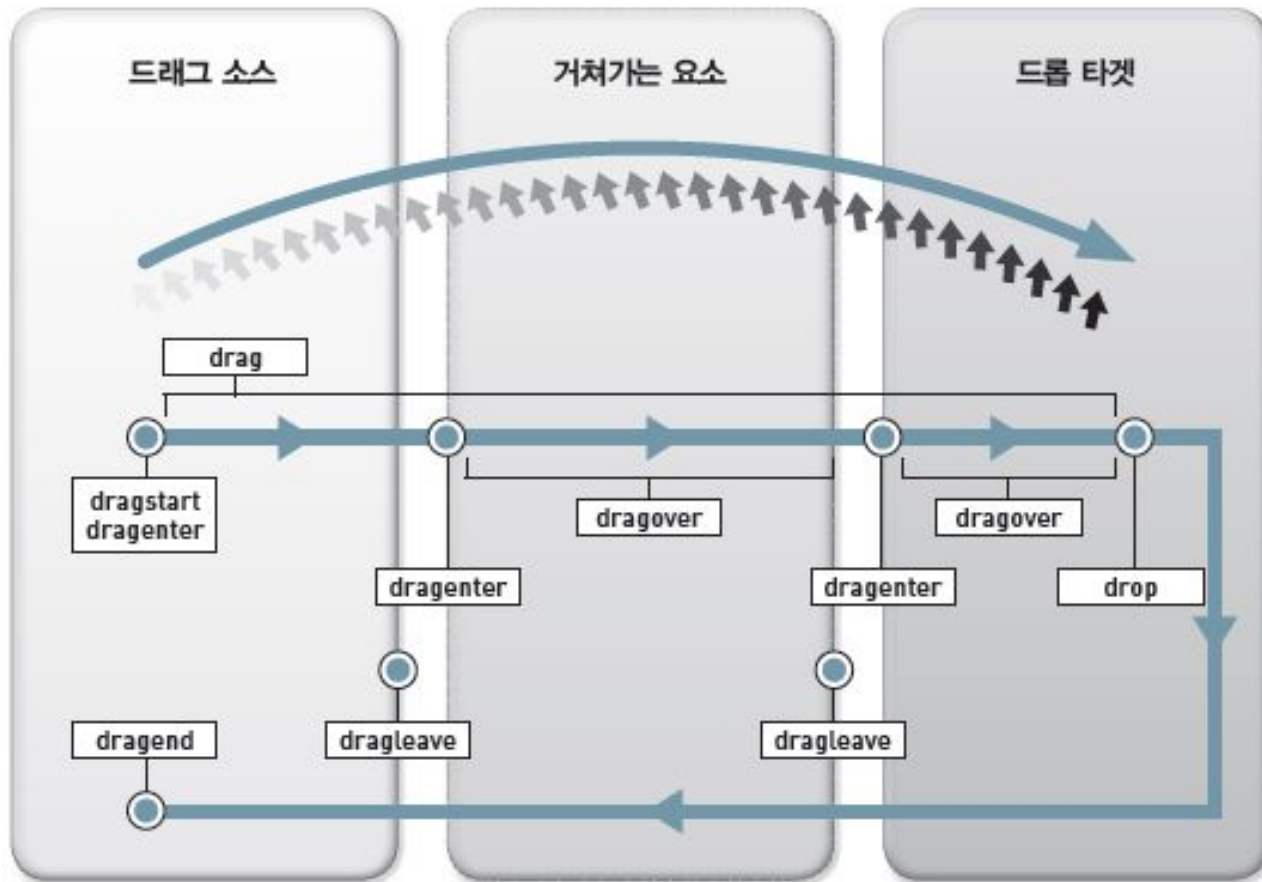


드래그



드롭

발생하는 이벤트

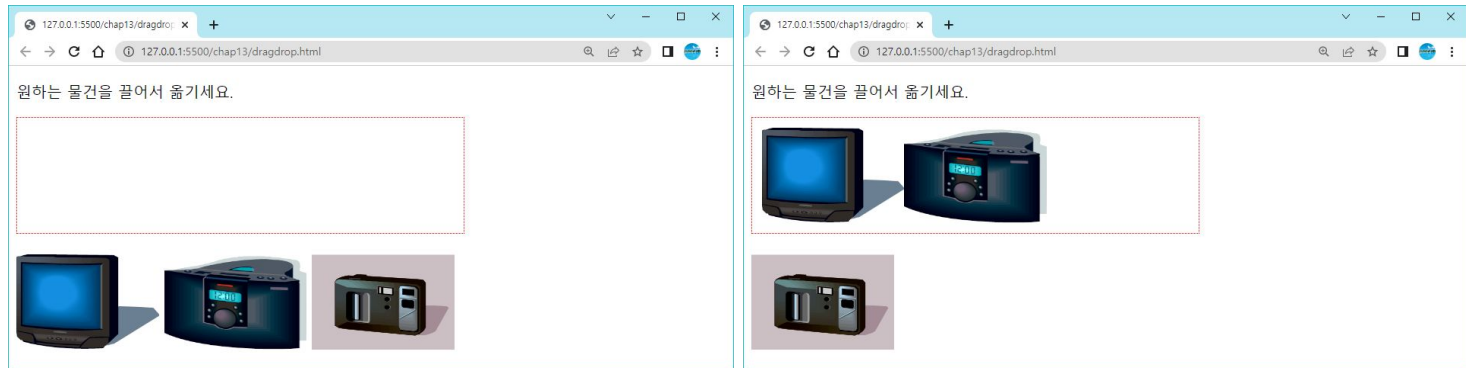


이벤트

- **dragstart** 이벤트: **dragstart** 이벤트는 사용자가 드래그를 시작할 때, 발생한다. 이때 해야 하는 가장 중요한 작업은 **dataTransfer** 객체에 **setData()** 호출을 통하여 데이터를 설정하는 작업이다.
- **drag** 이벤트: 드래그하는 도중에 계속해서 발생하는 이벤트이다.
- **dragenter** 이벤트: 마우스로 드래그하다가 새로운 요소 안으로 들어가면 **dragenter** 이벤트가 발생한다.
- **dragover** 이벤트: **dragover** 이벤트는 드래그 동작 도중에 마우스가 다른 요소 위에 있다는 것을 의미한다.
- **drop** 이벤트: 반드시 처리해야 하는 이벤트가 등장하였다. **drop** 이벤트는 사용자가 마우스 버튼을 놓았을 때, 발생한다. **dataTransfer** 객체에서 **getData()** 메소드를 이용하여서 필요한 데이터를 꺼내면 된다.

예제:

- 웹 페이지 위에 상품을 드래그하여 쇼핑 카트에 넣을 수 있도록 웹페이지를 작성하여 보자.



예제

```
<!DOCTYPE HTML>  
<html>  
  <head>  
    <style>  
      #shopping_cart {  
        width: 450px;  
        height: 100px;  
        padding: 10px;  
        border: 1px dotted red;  
      }  
    </style>  
    <script>  
      function allowDrop(e) {  
        e.preventDefault();  
      }  
  
      function handleDragStart(e) {  
        e.dataTransfer.effectAllowed = 'move';  
        e.dataTransfer.setData("Text", e.target.id);  
      }  
    </script>  
  </head>  
</html>
```

예 제

```
function handleDrop(e) {
    e.preventDefault();
    let src = e.dataTransfer.getData("Text");
    e.target.appendChild(document.getElementById(src));
}
</script>
</head>
<body>
    <p>원하는 물건을 끌어서 옮기세요.</p>
    <div id="shopping_cart"
        ondrop="handleDrop(event)" ondragover="allowDrop(event)"></div>
    <br>
    
    
    
</body>
</html>
```

HTML5 위치정보

- 위치정보 (**Geolocation**)은 자신의 위치를 웹 사이트와 공유
- 현재 지역의 날씨, 유명한 맛집 등의 정보를 제공받을 수 있다.



geolocation 객체

- `var geolocation = navigator.geolocation;`

메소드	설명
<code>getCurrentPosition()</code>	사용자의 현재 위치 정보를 반환한다.
<code>watchPosition()</code>	장치의 현재 위치에 대한 정보를 주기적으로 반환한다.
<code>clearWatch()</code>	현재 진행 중인 <code>watchPosition()</code> 실행을 중지한다.

getCurrentPosition() 메소드

- `getCurrentPosition()` 메소드는 위치 정보를 가져올 때 사용하는 메소드이다. 위치 정보를 처리하는 함수를 인수로 전달한다. 예를 들어서 다음과 같이 사용할 수 있다.

```
function getLocation() {  
  let geolocation = navigator.geolocation;  
  geolocation.getCurrentPosition(showLocation, errorHandler);  
}
```

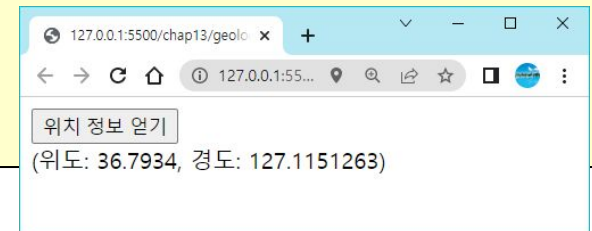
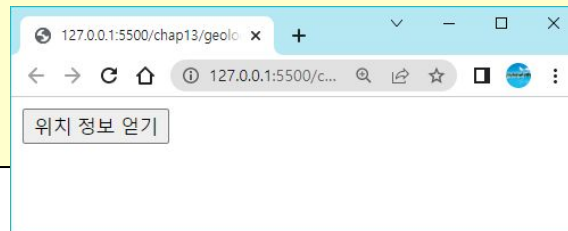
```
function showLocation( position ) {  
  let latitude = position.coords.latitude;  
  let longitude = position.coords.longitude;  
  ...  
}
```

position 인수

속성	타입	설명
coords	objects	장치의 위치정보를 가지고 있는 객체이다.
coords.latitude	Number	위도 정보이다. 단위는 도(decimal degree)이다. 범위는 [-90.00, +90.00].
coords.longitude	Number	경도 정보이다. 단위는 도(decimal degree)이다. 범위[-180.00, +180.00].
coords.altitude	Number	고도 정보이다. 단위는 미터이다.
coords.heading	Number	북쪽으로부터 시계방향으로 방위각

예제

```
<!DOCTYPE html>
<html>
<body>
  <button onclick="getGeolocation()">위치 정보 얻기</button>
  <div id="target"></div>
  <script>
    let myDiv = document.getElementById("target");
    function getGeolocation() {
      if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showLocation);
      }
    }
    function showLocation(location) {
      myDiv.innerHTML = "(위도: " + location.coords.latitude +
        ", 경도: " + location.coords.longitude + ")"
    }
  </script>
</body>
</html>
```

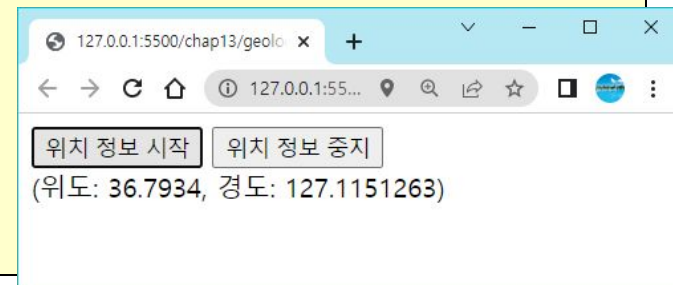


이동하면서 위치 정보를 얻는 방법

- 사용자가 이동하면서 주기적으로 위치정보를 얻으려면 **geolocation** 객체의 **watchPosition()**을 호출하여 콜백 메소드를 등록한다. 콜백 메소드가 주기적으로 호출되면서 업데이트된 위치정보를 얻을 수 있다.
- **watchPosition()** - 사용자의 현재 위치를 연속하여 출력한다. 자동차처럼 사용자가 이동하고 있으면 계속 업데이트된 위치를 반환한다.
- **clearWatch()** - **watchPosition()** 메서드를 중지한다.

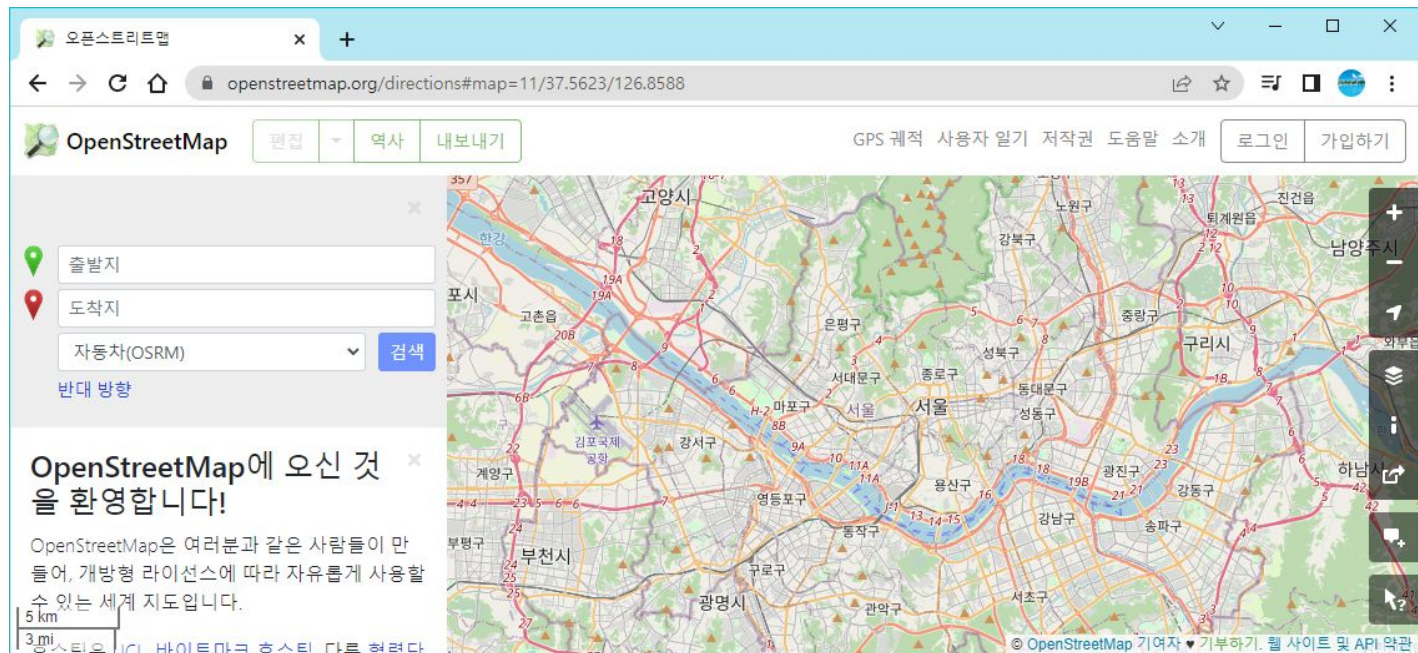
예제

```
<!DOCTYPE html>
<html>
<body>
  <button onclick="startGeolocation()">위치 정보 시작</button>
  <button onclick="stopGeolocation()">위치 정보 중지</button>
  <div id="target"></div>
  <script>
    let id;
    let myDiv = document.getElementById("target");
    function startGeolocation() {
      if (navigator.geolocation) {
        id = navigator.geolocation.watchPosition(showGeolocation);
      }
    }
    function showGeolocation(location) {
      myDiv.innerHTML = "(위도: " + location.coords.latitude +
        ", 경도: " + location.coords.longitude + ")";
    }
    function stopGeolocation() {
      if (navigator.geolocation) {
        navigator.geolocation.clearWatch(id);
      }
    }
  </script>
</body>
</html>
```



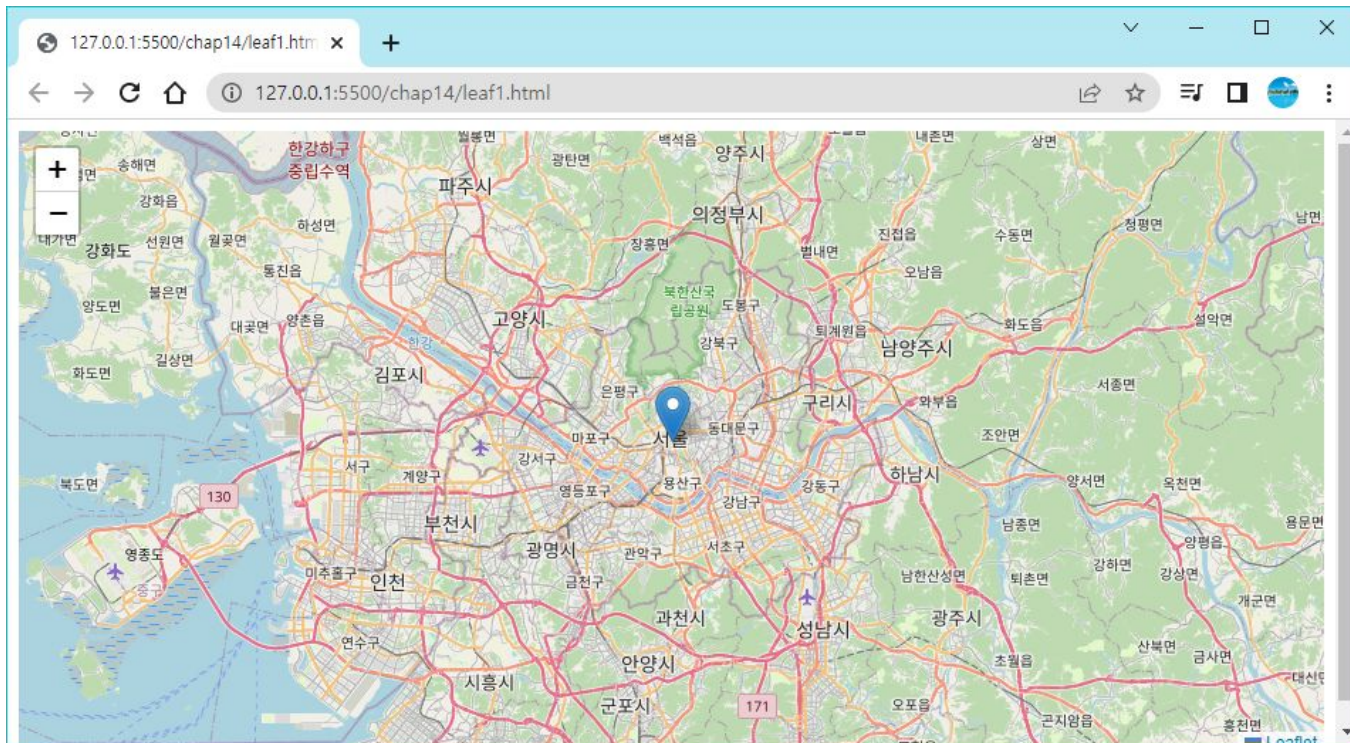
Open Street Map 지도 이용하기

- 지도에 결과를 표시하기 위해서는, 구글 맵(Google Map)처럼 위도 및 경도를 사용할 수 있는 지도 서비스에 대한 액세스가 필요하다.
- 오픈 지도 서비스인 **Open Street Map**을 사용해보자. 지도 타일 이미지는 **OpenStreetMap** 서버에서 가져온다.



예제

- 우리는 **Leaflet** 라이브러리를 통하여 **Open Street Map**을 사용해보자. 서울을 중심으로 지도를 표시하고 서울에 마커를 표시한다.



Open Street Map 지도 이용하기

- Leaflet CSS 소스를 포함하여야 한다.
- Leaflet 라이브러리 소스를 포함하여야 한다.
- 지도 컨테이너로 사용할 **<div>** 요소가 필요하다.
- 간단한 지도를 만드는 자바스크립트 코드가 필요하다.

예제

```
<html>
<head>
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.8.0/dist/leaflet.css"/>
  <script src="https://unpkg.com/leaflet@1.8.0/dist/leaflet.js"></script>
  <style>
    #map {
      width: 100%;
      height: 100vh;
    }
  </style>
</head>
<body>
  <div id="map"></div>
  <script>
    let mapOptions = {
      center: [37.562, 126.9793],
      zoom: 10
    }
    let map = new L.map('map', mapOptions);
    let layer = new L.TileLayer('http://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png');
    map.addLayer(layer);
    let marker = new L.Marker([37.562, 126.9793]);
    marker.addTo(map);
  </script>
</body>
</html>
```

HTML5 웹 워커

- 가끔은 자바스크립트에서 시간이 많이 걸리는 작업들을 할 때가 있다. 이런 작업들을 자바스크립트에서 한다면 웹 페이지는 자바 스크립트가 완료 될 때까지 응답 하지 않게 된다.
- **웹 워커(web worker):** 자바스크립트에 백그라운드에서 실행되는 스레드(thread)를 도입한 것



소수 구하기 예제

worker.js

```
// 소수를 찾는 자바스크립트 소스
let n = 1;
search: while (true) {
  n += 1;
  for (let i = 2; i <= Math.sqrt(n) ; i += 1)
    if (n % i == 0)
      continue search;
  // 소수를 발견할 때마다 바로 웹페이지로 전달한다.
  postMessage(n);
}
```

소수 구하기 예제

webworker.html

```
<!DOCTYPE HTML>
<html>
<head>
  <title>웹워커 예제</title>
</head>
<body>
  <button onclick="startWorker()">웹워커 시작</button>
  <button onclick="stopWorker()">웹워커 종료</button>
  <p>현재까지 발견된 가장 큰 소수는
    <output id="result"></output>
  </p>
```

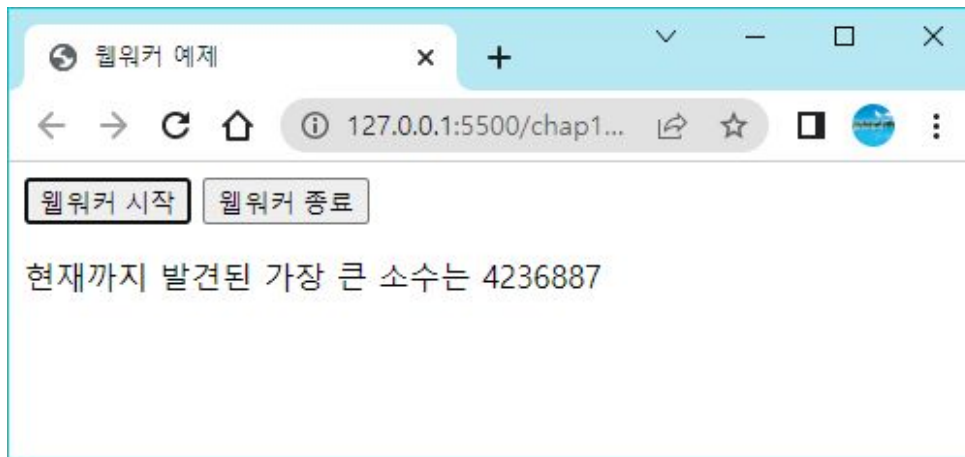
소수 구하기 예제

webworker.html

```
<script>
  let w;
  function startWorker() {
    if (typeof (Worker) !== "undefined") {
      if (typeof (w) == "undefined") {
        w = new Worker("worker.js");
      }
      w.onmessage = function (event) {
        document.getElementById("result").innerHTML = event.data;
      };
    }
    else {
      document.getElementById("result").innerHTML = "웹브라우저가 웹워커를
지원하지 않음";
    }
  }
  function stopWorker() {
    w.terminate();
  }
</script>
</body>
</html>
```

소수 구하기 예제

- 이번 예제는 반드시 웹서버를 통하여 실행하여야 한다. 즉 라이브 서버를 통하여 실행하여야 한다. 단순히 더블 클릭만 하면 웹워커가 실행되지 않는다.

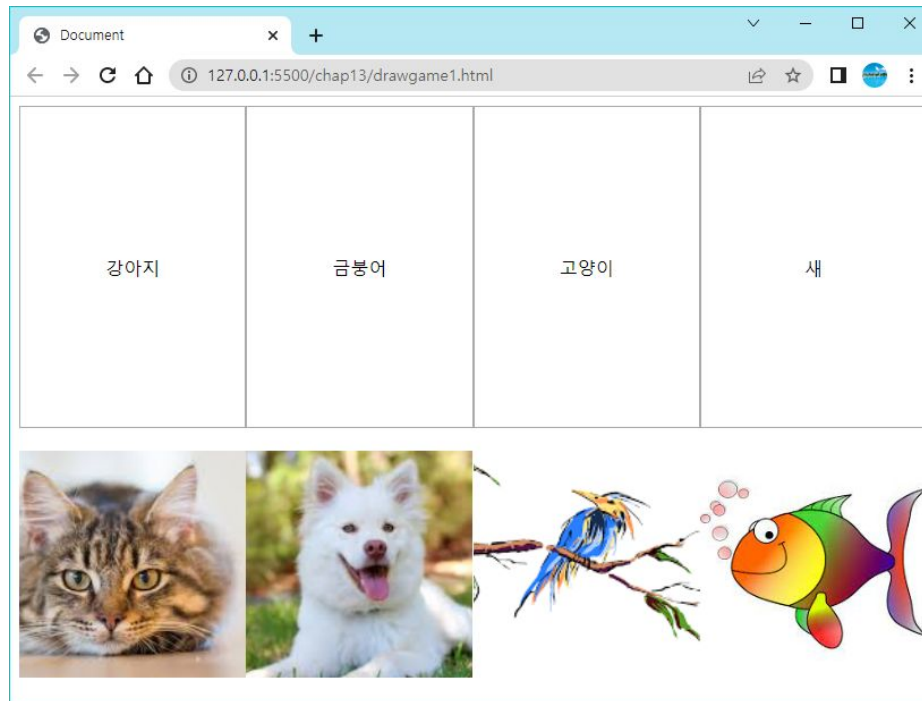


웹워커의 용도

- 영상 처리(image processing) - 캔버스나 비디오 요소에서 추출된 데이터를 사용하여 어떤 영상 처리를 수행할 수 있다. 이때는 이미지를 여러 개의 작은 조각으로 나누어서 각 조각들을 웹워커에게 줄 수도 있다. 멀티-코어 CPU라면 물리적으로도 웹워커들이 동시에 수행될 수 있다.
- 대용량 데이터 처리 – XMLHttpRequest 호출 후에 파싱해야할 대용량 데이터가 존재하는 경우에 사용할 수 있다.
- 텍스트 분석 – 사용자가 입력하는 즉시 텍스트를 사전에서 검색한다. 자동적인 오류 수정 등이 가능하다.
- 데이터베이스 요청 동시 수행 – 로컬 데이터베이스에 대한 요청들을 동시에 수행한다.

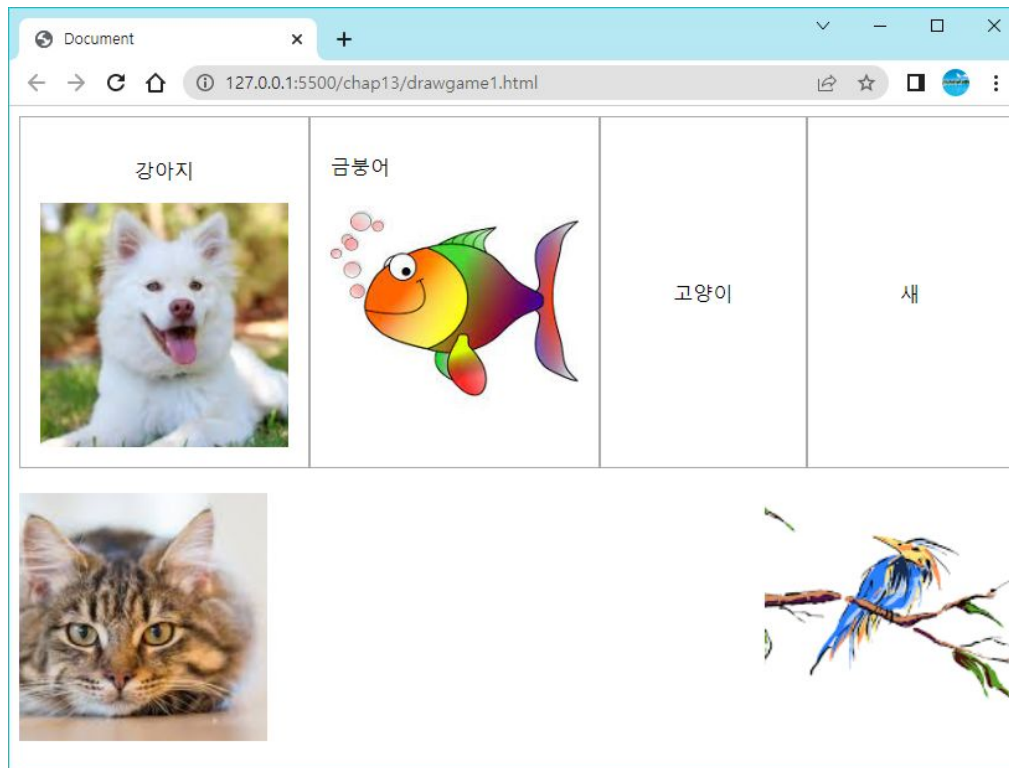
Mini Project: 맞추기 게임

- 드래그와 드롭을 이용하여서 상단의 텍스트 박스와 하단의 이미지를 일치시키는 게임을 작성해보자.



Mini Project: 맞추기 게임

- 드래그와 드롭을 이용하여서 상단의 텍스트 박스와 하단의 이미지를 일치시키는 게임을 작성해보자.



Q & A

